Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025

Lab. 7.3. POO. Polimorfismo en Python

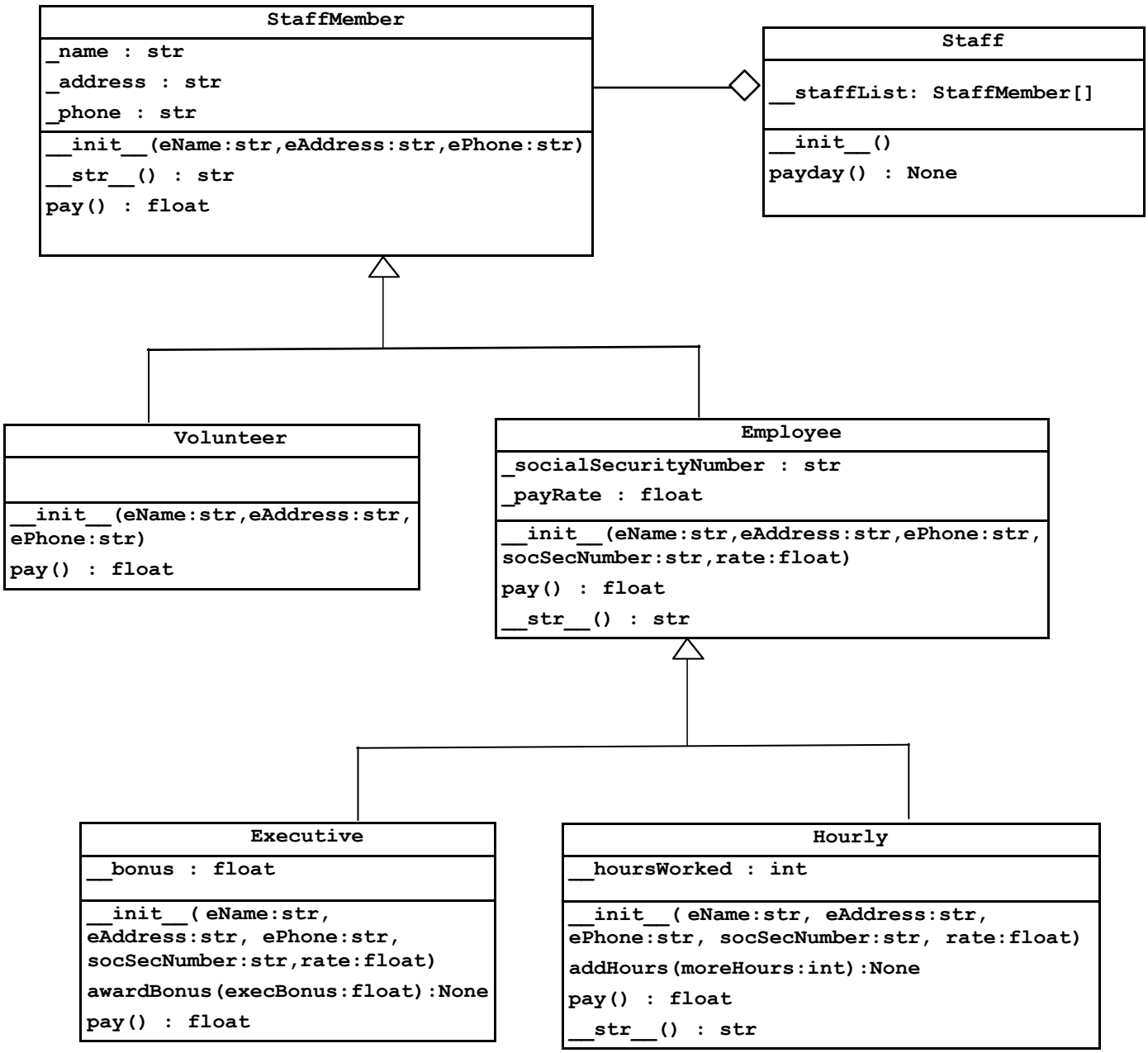
Octubre 04, 2025

Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Implementación de Polimorfismo en Python

Se dice que un objeto es polimórfico cuando su comportamiento varía de acuerdo a un contexto determinado. Python es un lenguaje polimórfico, por lo que el concepto de polimorfismo es inherente al lenguaje. ¿Porque se dice que Python es un lenguaje polimórfico?

1. Implemente la siguiente jerarquía de clases. Siga las recomendaciones.



Inicie con la implementación de la clase `StaffMember`, esta clase es especial, es la clase base para todos los tipos de trabajadores. Representa a un trabajador en general, por lo que el método `pay()` no realiza tarea alguna, debido a que no representa a un tipo de trabador en específico, cada tipo de trabajador tiene su propia forma de calcular su pago, es decir, tiene su propio `pay()`.

En la clase `Employee`, el método `pay()` basta que devuelva el campo `payRate`.
En la clase `Volunteer`, basta que `pay()` devuelva 0.0 debido a que un trabajador voluntario normalmente no percibe sueldo alguno.

La clase `Executive` tiene un campo adicional `bonus`. Tener en cuenta que no todos los trabajadores tienen bono. El trabajador recibe bono de manera explícita mediante el método `awardBonus()`, su sueldo final debe incluir el bono si lo hubiera.

La clase `Hourly` tiene el método `addHours()` que simplemente acumula las horas del trabajador en el campo `hoursWorked`. El pago se calcula multiplicando `payRate` por `hoursWorked`.

Implemente la clase `Staff`, esta clase representa al total de trabajadores de la empresa. El inicializador crea al menos 6 objetos, un `Executive`, dos `Employee`, un `Hourly`, y dos `Volunteer`. El método `payday()` muestra el pago de cada trabajador de la empresa, para lo cual hace uso del método `pay()` de cada objeto. Fíjese que `pay()` está implementado en cada clase según el tipo de trabajador. Note que `Staff` usa composición en lugar de herencia.

2. Escriba un programa para probar la clase Staff. Genere un objeto Staff que represente a los trabajadores de la empresa, muestre el pago de cada trabajador y sus datos personales.

3. Muestre un listado con los nombres y las direcciones de todos los trabajadores.

4. Determine al trabajador con el mayor sueldo.

5. Elabore una tabla donde se muestre, en cada fila, el tipo de trabajador y su cantidad.

OPCIONAL

6. El siguiente código implementa cuatro clases y proporciona el programa de prueba. Coloque el código en un archivo. Ejecute el programa.

```
class Vehicle:
    def __init__(self, model):
        self.model = model

   def vehicle_model(self):
        print(f"Vehicle Model name is {self.model}")

class Bike(Vehicle):
    def vehicle_model(self):
        print(f"Vehicle Model name is {self.model}")

class Car(Vehicle):
    def vehicle_model(self):
        print(f"Vehicle Model name is {self.model}")

class Aeroplane:
    pass
```

```
# programa Test

def vehicle_info(vehicle_obj):
    vehicle_obj.vehicle_model()

def main():
    ducati = Bike("Ducati-Scrambler")
    beetle = Car("Volkswagon-Beetle")
    boeing = Aeroplane()
    for each_obj in [ducati, beetle, boeing]:
        try:
            vehicle_info(each_obj)
        except AttributeError:
            print("Expected method not present in the object")

if __name__ == "__main__":
    main()
```

a) ¿Qué parte del código enfatiza el concepto de polimorfismo?

b) ¿Tiene el objeto `ducati` el atributo `model`?, si lo tiene ¿cómo lo inicializa?.

c) Analice el caso, ¿tiene sentido instanciar un objeto de la clase `Vehicle`?

7. ¿Cuál es la salida del siguiente código? Explique su respuesta.

```
class A:
    def __str__(self):
        return '1'

class B(A):
    def __init__(self):
        super().__init__()

class C(B):
    def __init__(self):
        super().__init__()


def main():
    obj1 = B()
    obj2 = A()
    obj3 = C()
    print(obj1, obj2,obj3)

main()
```

a) 1 1 1

b) 1 2 3

c) '1''1''1'

d) Se lanza una excepción (mensaje de error)

8. ¿Cuál es la salida del siguiente código? Explique su respuesta.

```
class Demo:
    def __init__(self):
        self.x = 1

    def change(self):
        self.x = 10

class Demo_derived(Demo):
    def change(self):
        self.x=self.x+1
        return self.x


def main():
    obj = Demo_derived()
    print(obj.change())

main()
```

a) 11

b) 2

c) 1

d) Se lanza una excepción (mensaje de error)

9. ¿Cuál es la salida del siguiente código? Explique su respuesta.

```
class A:
    def __repr__(self):
        return "1"

class B(A):
    def __repr__(self):
        return "2"

class C(B):
    def __repr__(self):
        return "3"


o1 = A()
o2 = B()
o3 = C()

print(obj1, obj2, obj3)
```

a) 1 1 1

b) 1 2 3

c) '1''1''1'

d) Se lanza una excepción (mensaje de error)

10. Analice el siguiente código e indique la respuesta correcta. Explique su respuesta.

```
class A:
    def __init__(self):
        self.setI(20)
        print("i from A is", self.i)

    def setI(self, i):
        self.i = 2 * i;

class B(A):
    def __init__(self):
        super().__init__()

    def setI(self, i):
        self.i = 3 * i;

b = B()
```

A. El método __init__ de la clase A no se invoca.

B. El método __init__ de la clase A se invoca y muestra `"i from A is 0"`.

C. El método __init__ de la clase A se invoca y muestra `"i from A is 40"`.

D. El método __init__ de la clase A se invoca y muestra `"i from A is 60"`.

Guarde todos vuestros programas en una carpeta con el nombre su **Apellido** paterno seguido de vuestro **DNI**, luego comprima esta carpeta. Envíe este archivo a Katherine Navarro katherine.navarro@upch.pe especificando como asunto **Lab7.3**.